CS496 Software Project: VR Puzzle Game

Joe Fielding and Kyle Richards

2026-04-17

1 Client Information

By sharing this client information and the rest of this document, you are stating that this client has provided this project as something they want (not something you created and asked if they wanted), and that they are interested in having you complete this project for your capstone.

- Client name: Hoang Bui
- Client title: Associate Professor
- Client email address: hdbui@loyola.edu
- Client employer: Loyola University Maryland
- How you know the client: We had him as a professor, have work with him, and he is our Competitive Programming Coach

2 Project Description

2.1 Overview

The goal of this project is to create a virtual reality (VR) puzzle game that can be played using the Meta Quest headset. This app will allow the user to take pictures of real-life objects, and then the app will generate a puzzle by breaking down the real-life object into pieces. The pieces could either be normal jigsaw puzzle shaped pieces or different shapes of the user's choosing. The user should also be able to adjust the difficulty of the puzzle by choosing the number of pieces. Finally, the user can import the puzzle into the VR world and attempt to solve it by fitting the pieces together by using their hand motions.

The problem that this project is trying to solve is that the issue with real jigsaw puzzles is that you must buy another one every time you want to solve a new one. This is impractical for Dr. Bui because he enjoys solving puzzles but does not have the time to keep going out and buying new ones. He wants to be able to generate new puzzles quickly and then solve them using his Quest headset. This app will solve this issue because Dr. Bui can take a picture of any real object of his choosing, and then the app will generate a puzzle for him.

Another issue with real life puzzles is that they become less interesting if you are to do them a second time because you already know the solution. Since the app can generate new puzzles on the fly, it will keep Dr. Bui engaged by giving him a new challenge.

2.2 Key Features

- Scan picture of real object into puzzle, Solve the puzzle in VR world
- Ability to choose the number and shape of the puzzle pieces

- Since the puzzles may be challenging and take some time, Dr. Bui wants to be able to keep track of his progress on a puzzle so he can come back to it later
- Keep track of how long it takes to solve a particular puzzle. This allows for competition as you can challenge someone else to solve the same puzzle
- Leaderboard displaying the best solving times for a particular puzzle
- Providing hints for where the pieces fit
- Additionally, Dr. Bui wanted to have the ability to be able to take a picture of a real puzzle and then import it into the VR world. This would still use the same pieces, so essentially just solving the same puzzle but it is instead while being in the VR world.
- This app would also have to figure out how to solve the real puzzle in order to provide hints. Giving hints for the real life objects is much easier as you know the solution before you break it into a puzzle.

2.3 Why this Project is Interesting

This project is interesting because it is unique compared to many other senior capstone projects. There is also learning to be done to figure out how to make this app work, which we are excited about as we have never done a project related to VR. Additionally, it seems there is not much existing support for scanning real life objects and converting them into puzzles. Most mainstream puzzle apps use premade objects (such as the one linked below). Some of them did have an uploading feature, but it was a 2D image rather than taking a picture of a 3D object in real life.

Existing Puzzle Game Link: `https://www.meta.com/experiences/art-puzzle/5920812271341547/`

2.4 Areas of CS required

- Human Computer Interaction
- Game Development
- Algorithms
- Computer Vision

2.5 Potential Concerns and Questions

We think this project should fit the requirement of a senior capstone as there are several different areas of CS involved and the project will certainly be a lot of work. The only concern we would have for ourselves is that a lot of this will be new to us. We are not sure yet about how to use the hardware to accomplish all of this. Despite this, we are good at learning new technologies and ideas, so we should be able to manage well.

2.6 Summary of Efforts to Find a Project

We talked to Dr. Bui in person about this project, and he gave us the requirements and explained why he wanted this project. We have another project that we are currently considering, which is a SWE project for Blue Raven. We are currently a little concerned that the Blue Raven project may not have enough to do for a senior capstone which is why we are continuing to pursue multiple projects in parallel.

2.7 Comparison to Draft

This is not the same project that either of us proposed. We had concerns about the original project that we were trying to do because it seemed that it might be too basic. The original project was the one with Blue Raven. We have had struggles with getting in contact with the people who would help us refine the requirements of the project. This project definitely seems to be more promising. We decided to pursue both a completely different project and client due to the same concerns above. We were considering asking for a new project, but we were afraid of having the same issues of communication with them.

3 Requirements

3.1 Non-Functional Requirements

ID	NFR Title	Category	Description
NFR1	Puzzle Generation Speed	Performance	Puzzle should be generated quickly in less than 3 seconds
NFR2	Framerate	Performance	Game should run at at least 72fps (equal to minimum refresh rate) on the meta quest headset
NFR3	Understandable Menus	Usability	Good UI/UX for menus, easy to understand what menus are for
NFR4	Understandable Interactions	Usability	Pieces are easy to interact with and interactions are intuitive
NFR5	Game Server	Reliability	Server should be almost always up so that users can attempt uploaded puzzles and view the leaderboards at any time

Table 1: Non-Functional requirements

3.2 Functional Requirements (User Stories)

Table 2: Functional requirements as User Stories.

ID	Story Title	Points	Description
U0	Piece Movement	1	As a user I want to be able to pick up and move around puzzle pieces using my real hand motions so that I can figure out how to solve the puzzle.
U1	Piece Connection	5	As a user, I want to be able to connect puzzle pieces together so that I can make progress towards solving the overall puzzle.
U2	2D Picture Upload	1	As a user, I want to be able to upload a picture file contains a 2D object so that I can solve a puzzle consisting of pieces of that object.
U3	Jigsaw Piece Generation	8	As a user, I want to be able to generate jigsaw puzzle pieces out of an object so that I can reconstruct the object by fitting the pieces back together.
U4	Local Puzzle Creation	2	As a user I want to be able to create a puzzle locally so that I can solve it on my headset.
U5	Local Puzzle Delete	2	As a user I want to be able to remove a locally created puzzle so that I can get rid of puzzle that I don't want to play anymore.
U6	Move Puzzle	2	As a user I want to be able to pick up my partially solved puzzle and move it so that I can reorganize my workspace while solving the puzzle.
U9	Solve Time	2	As a user I want to be able to see how long it takes me to solve a puzzle so that I can put my puzzle solving skills to the test and challenge my friends.
U10	Puzzle Upload	2	As a user I want to be able to upload puzzles that I create so that other people can also attempt to solve them.
U11	Uploaded Puzzle Deletion	2	As a user I want to be able to delete puzzles that I uploaded so that I change my mind about which of my puzzles I want publicly available.
U12	View Other Puzzle Uploads	2	As a user I want to be able to view puzzles created by other people so that I can decide which ones I want to try to solve.

Continued on next page

Table 2 – *Continued from previous page*

ID	Story Title	Points	Description
U13	Edit Uploaded Puzzles	2	As a user I want to be able to edit the puzzles that I uploaded myself so that I can make changes when I want to.
U14	Save Progress	2	As a user I want to be able to save my current progress on a puzzle so that I can come back and finish it later.
U15	Provide Piece Hint	2	As a user I want to be able to receive a hint as to where one of the pieces that I have not yet connected fits into the puzzle so that I can get help when I am stuck.
U17	Leader Board Creation	2	As a user I want to have a leader board generated after I upload a puzzle so that I can see who is the best at solving my puzzle.
U18	View Puzzle Leader Board	2	As a user I want to be able to view the leader boards of uploaded puzzles so that I can see where my time ranks compared to others.
U19	Leader Board Update	2	As a user I want the leader board to update my I or someone else solves the puzzle so that I can see the most up to date standings.
U20	Take Picture with Headset	2	As a user I want to be able to take a picture of an object using the headset so that I can generate a puzzle from that image to solve.
U21	Crop Image on Headset	2	As a user I want to be able to crop an image that I take with the headset so that I can generate the puzzle out of only parts of the image I want to keep.
U22	Load Progress	2	As a user I want to be able to load my current progress on a puzzle so that I can come back and finish it.
U23	Piece Rotation	1	As a user I can rotate the puzzle pieces using my real hand motions so that I can orient them correctly when solving the puzzle.
U24	Real Puzzle Piece Detection	8	As a user I want the pieces of a real puzzle to be detected from an image so that I can solve the same puzzle but instead in VR.
U25	Real Puzzle Piece Detection Report	2	As a user I want the application to display how many pieces and the shapes of the pieces it detected so that I can know if the pieces are detected correctly.
U26	Real Puzzle Piece Render	5	As a user I want the puzzle pieces in the image to render the same in VR game so that I can solve the same puzzle expect in VR.
U27	User Auth	2	As a user I want to be authenticated so that I can access my own puzzles and not have others access them.
U28	Support Other Piece Shapes	5	As a user I want to be able to pick other piece shapes besides jigsaw so that I can complete puzzles of a variety of shapes.

4 System Design

4.1 Architecture

We will be using the MVC architecture with a Spring Boot backend, and Unity will handle all the views on the frontend. We chose to use the Web MVC architecture because we need to handle online puzzles which will involve uploading puzzles, and playing puzzles made by others.

The main modules of our software on the backend will be the config, controller, model, repository, security, service, and puzzle generation modules.

The purpose of the config module will simply be for configuration. For example, we are planning on using sockets for communication between the backend and frontend when a user is solving an online puzzle.

The purpose of the controller module is to define all the endpoints that will be called from the frontend, and it will additionally delegate tasks to the appropriate service module. For example, when a player requests to download a puzzle, after that request has been authenticated it will pass it off to the PuzzleController. The PuzzleController will then pass responsibility off to the Puzzle Service which will later return what needs to be sent back to the client. The Controller can then format this into a http response to send back.

The service module handles all of the business logic for requests. Using the example of downloading a puzzle, the Puzzle service will know that it needs to download the file associated with that puzzle and it will then delegate that task to the Content service.

The model module is split up into the dto, entity, and request modules. The dto represent the responses sent from the server to the client, the entity represents what is stored in the database, such as the Puzzle metadata, and the request module has all the formats of the requests sent from the client to the server.

The repository module piggybacks off of JPA and hibernate and handles all the interactions with the database.

The security module will define who is allowed to access what endpoints. It is also responsible for defining the filter that requests will go through before being passed to the controller for authorization purposes.

Finally, the puzzle generation module will do all the heavy lifting for both breaking down a puzzle into pieces, and figuring out how a puzzle fits back together in the case of a real world import. This is where the PuzzleGame class will live and it will store an array of the generated pieces after breaking down the object.

The main modules on the frontend will at least for now be a game representation module and a scripts module.

The game representation will store the low level representation of the puzzle game. This module will correspond to the puzzle generation module on the backend in terms of it having the same level of abstraction as they are both the lower level representation. This is where the logic of keeping track of which pieces are connected and which are still unconnected as the use works to solve the puzzle. We are thinking at this stage that this will involve making sets of connected pieces and when the user connects a piece, it would call the set union operation.

The scripts module will contain everything unity specific such as the scripts for handling the physics of moving and rotating puzzle pieces. This will also involve managing creating and deleting unity objects and managing parent and child relationships between them as pieces are connected.

4.2 Diagrams

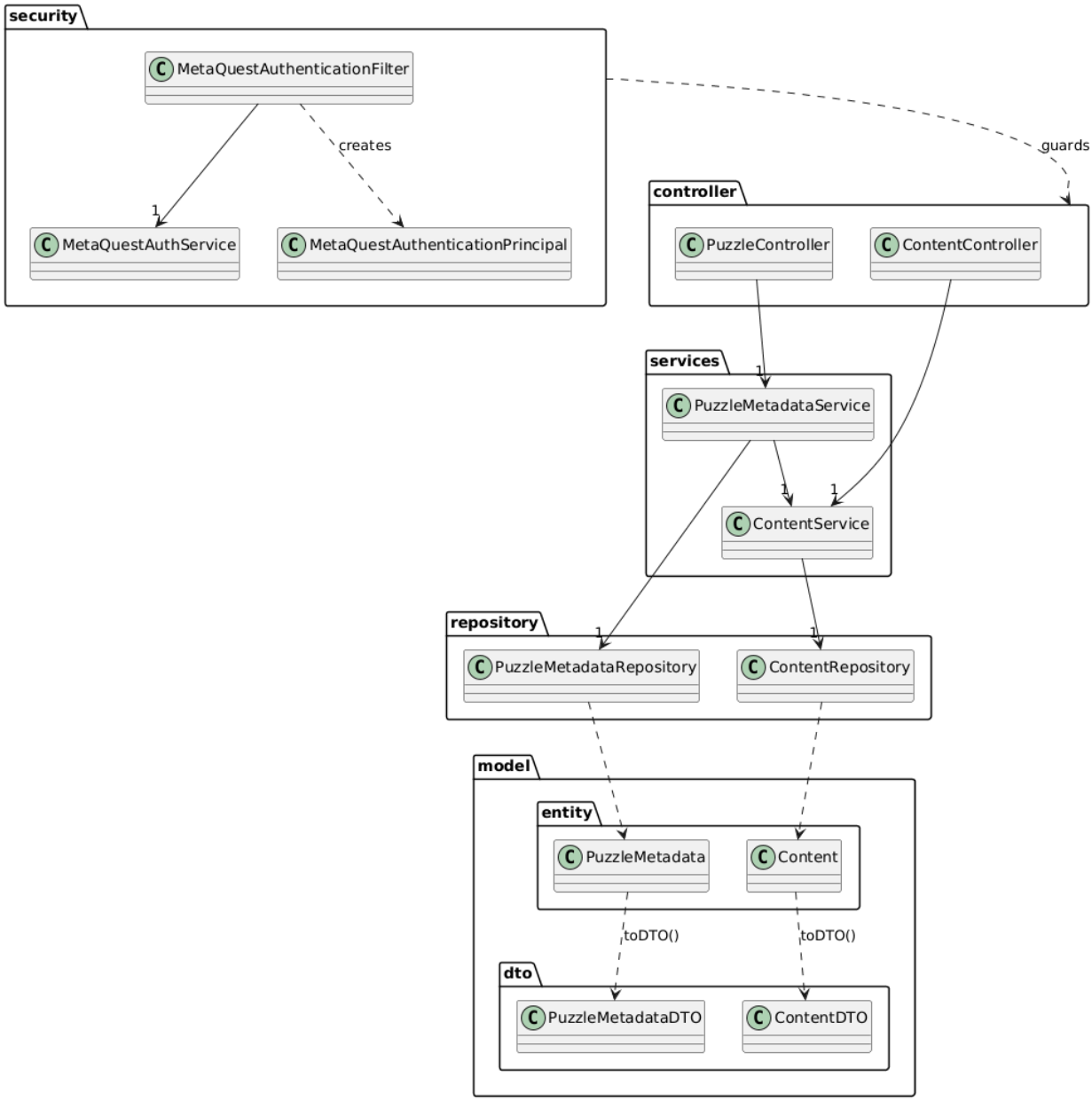


Figure 1: Current Backend Simplified Class Diagram

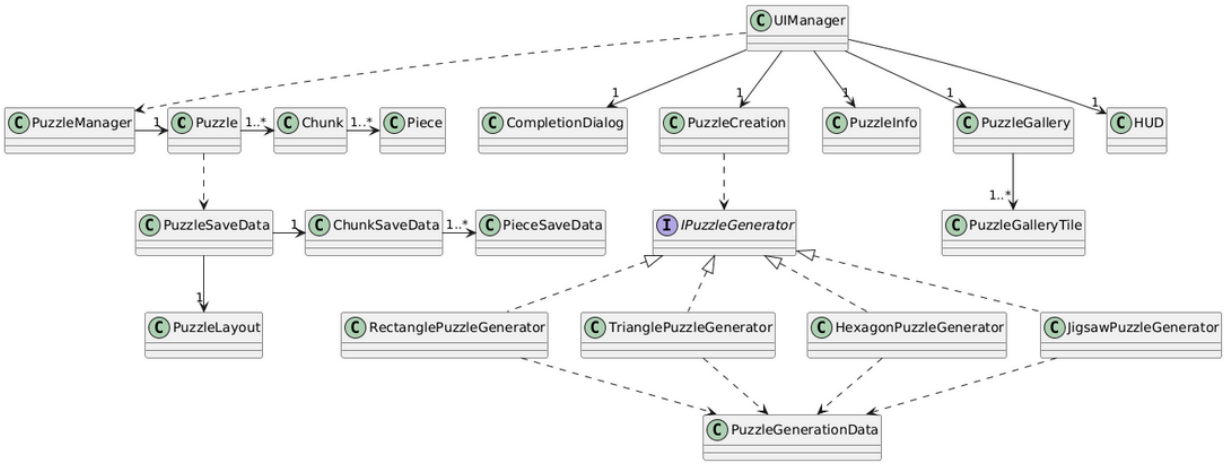


Figure 2: Current Unity Simplified Class Diagram

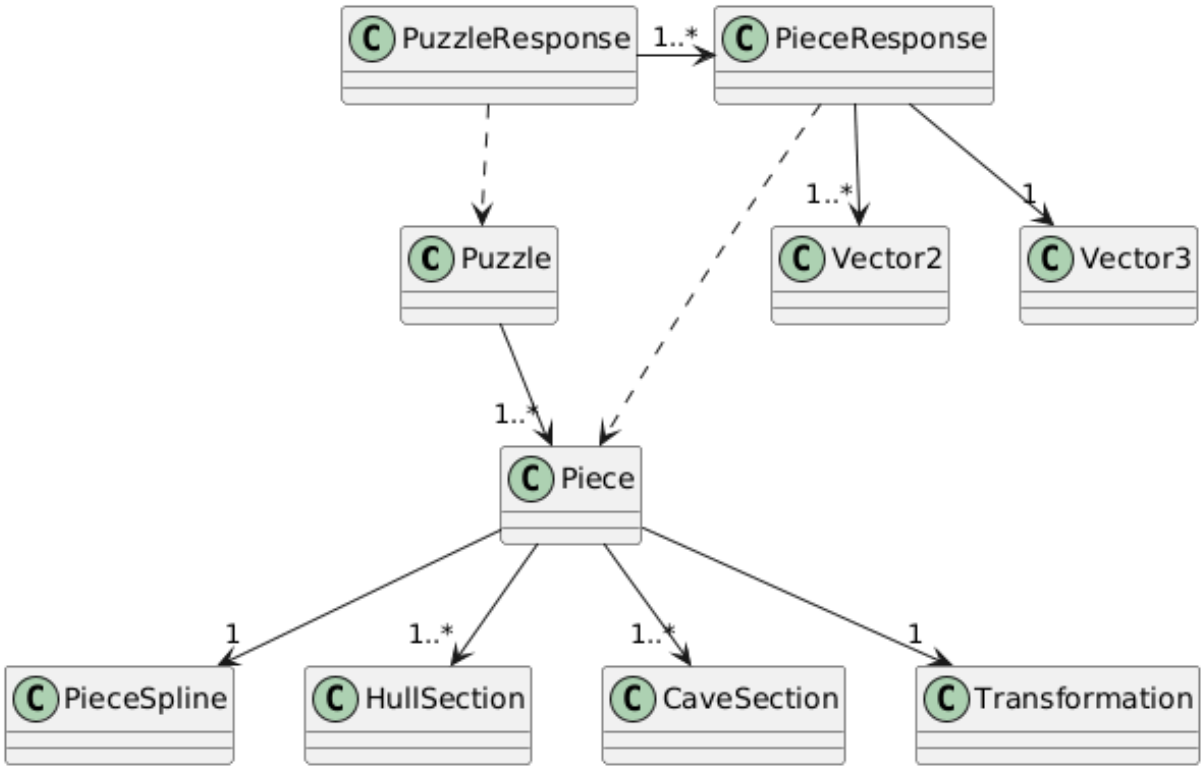


Figure 3: Current Real Puzzle Micoservice Simplified Class Diagram

4.3 Technology

Backend

- MongoDB

- Kotlin
- Spring Framework
- Spring Testing Framework

Frontend

- C#
- Unity 6
- Unity Testing Framework
- OpenXR
- Meta Quest XR All-In-One SDK, including:
  - Meta XR Core SDK
  - Meta XR Audio SDK
  - Meta XR Haptics SDK
  - Meta XR Interaction SDK Essentials
  - Meta XR Interaction SDK
  - Meta XR Platform SDK
  - Meta XR Simulator
  - Meta Mixed Reality Utility Kit

4.4 Coding Standards

- Database fields have snake case
- Database table names have Pascal Case
- C# codebase will use the official C# naming standards defined here:
  - https://learn.microsoft.com/en-us/dotnet/csharp/fundamentals/coding-style/identifier-names
- Kotlin codebase will use camel case for identifiers and Pascal case for classes
- Code needs at least 60% statement coverage to be merged into the main branch

4.5 Data

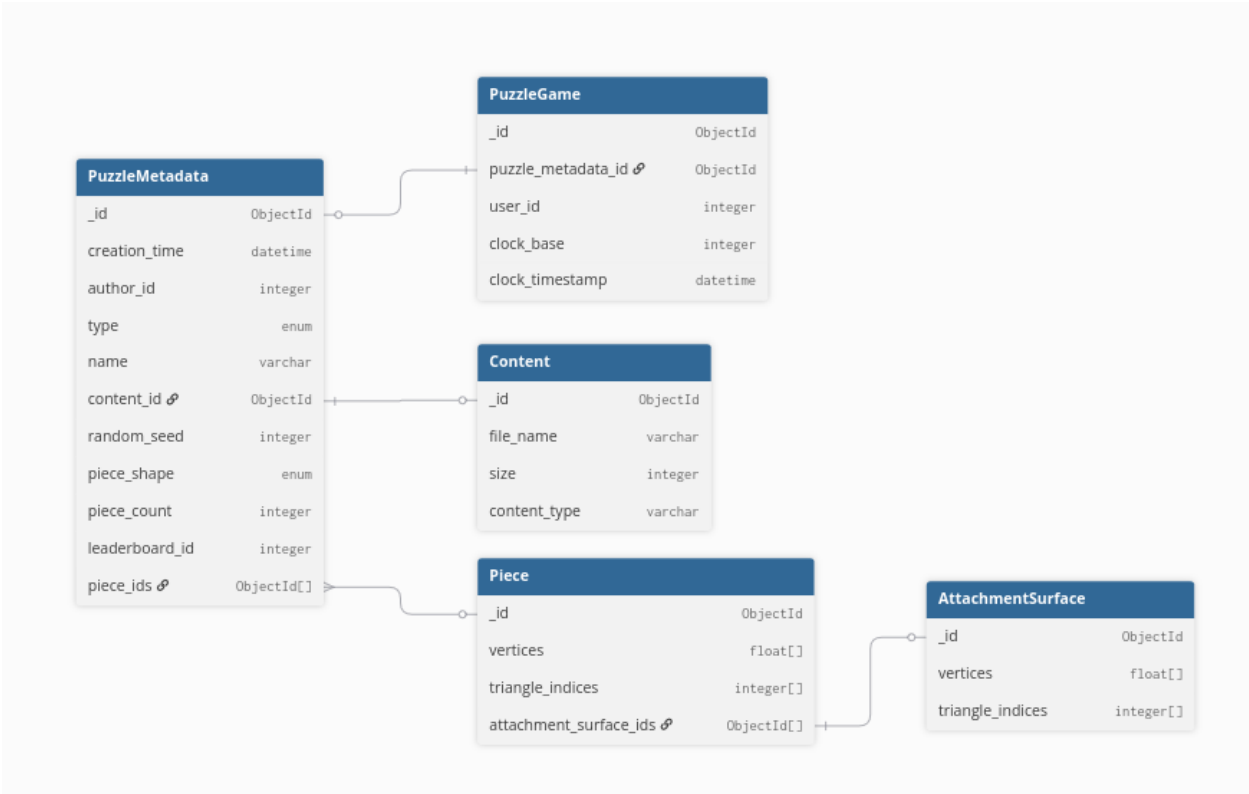


Figure 4: The reason that we do not have a Leaderboard table is that we found that Meta Quest has an API specifically for you and Meta can store these leaderboards.

4.6 UI Mocks

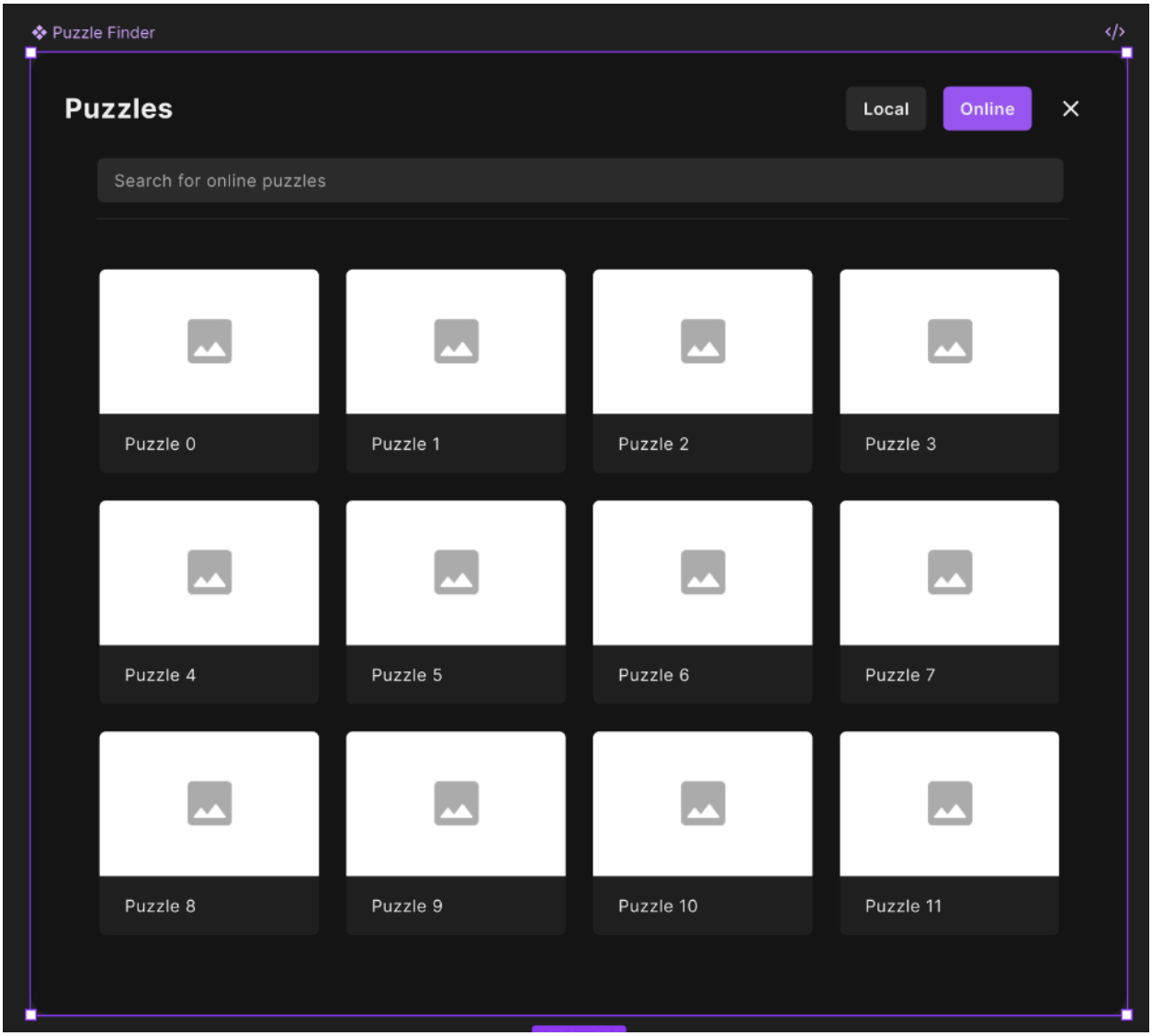


Figure 5: View of Puzzle List, with tabs for online and local

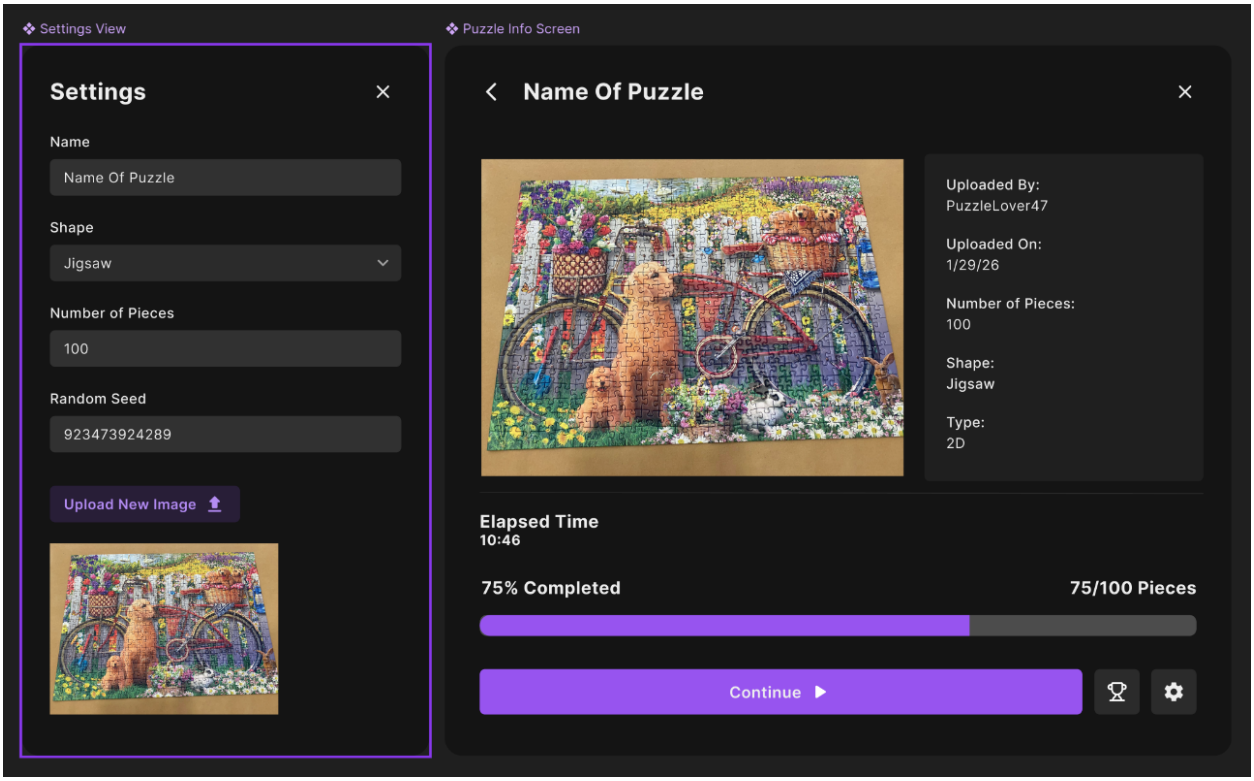


Figure 6: Puzzle Settings View

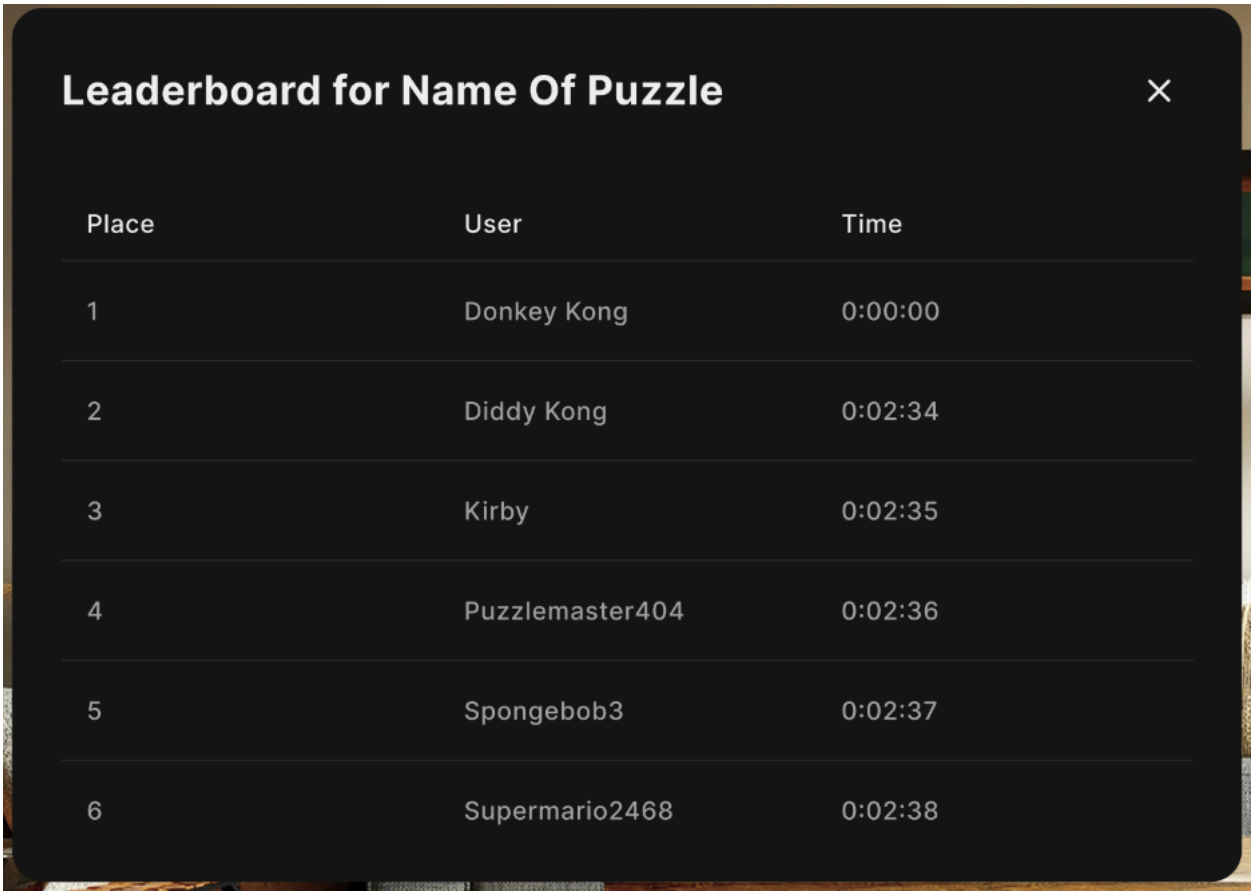


Figure 7: Puzzle Leaderboard View

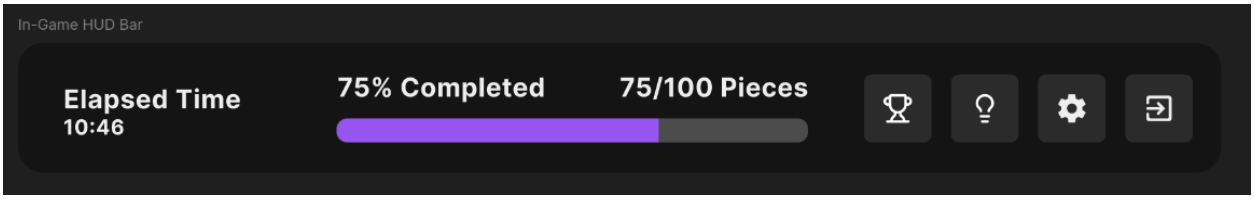


Figure 8: Hud that shows up on the top of the screen as the user is solving the puzzle

5 Iterations

5.1 Iteration Planning

Iteration	Dates	Stories	Points
1	01/27 - 02/10	U0 Piece Movement (1), U23 Piece Rotation (1), U6 Move Puzzle (1), U1 Piece Connection (5), U2 2D Picture Upload (1)	9
2	02/10 - 02/24	U3 Jigsaw Piece Generation (8) U4 Local Puzzle Creation (2), U5 Local Puzzle Delete (2), U14 Save Progress (2), U22 Load Progress (2)	16
3	02/24 - 03/17	U24 Real Puzzle Piece Detection (8), U25 Real Puzzle Piece Detection Report (2), U26 Real Puzzle Piece Render (5)	15
4	03/17 - 03/31	U10 Upload the puzzle (2) U11 Uploaded Puzzle Deletion (2) U12 View Other Puzzle Uploads (2) U13 Download puzzle (2) User auth(2) U15 Provide Piece Hint (3) Support generating other pieces besides just jigsaw (5)	18
5	03/31 - 04/14	U9 Solve Time (2), U17 Leader Board Creation (2), U18 View Puzzle Leader Board (2), U19 Leader Board Update (2), U20 Take Picture with Headset (2), U21 Crop Image on Headset (2)	12
		Total:	70

Table 3: Iteration Planning for Incremental Deliveries

5.2 Iteration/Sprint 1

5.2.1 Planning

Team Member	Story	Story Description	Points
Joe	U1	Piece Connection (Pair Programming)	2.5
	U23	Piece Rotation	1
	U6	Move Puzzle	1
		Joe Total	**4.5**
Kyle	U0	Piece Movement	1
	U1	Piece Connection (Pair Programming)	2.5
	U2	2D Picture Upload	1
		Kyle Total	**4.5**
		Total	**9**

Table 4: Iteration 1 Planning

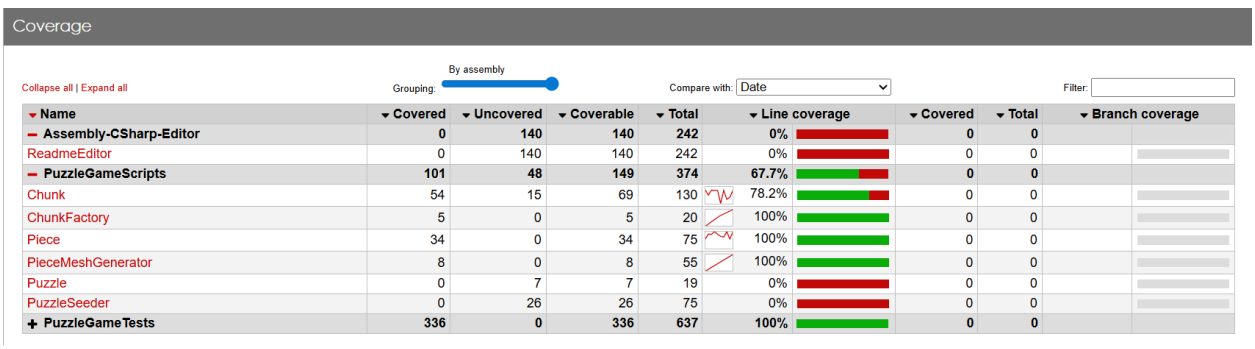
5.2.2 Work Done

Team Member	Story	Story Description	Points
Joe	U0	Piece Movement (Pair Programming)	0.5
	U1	Piece Connection (Pair Programming)	2.5
	U23	Piece Rotation (Pair Programming)	0.5
	U6	Move Puzzle (Pair Programming)	0.5
		Joe Total	**4**
Kyle	U0	Piece Movement (Pair Programming)	0.5
	U1	Piece Connection (Pair Programming)	2.5
	U23	Piece Rotation (Pair Programming)	0.5
	U6	Move Puzzle (Pair Programming)	0.5
		Kyle Total	**4**
		Total	**8**

Table 5: Iteration 1 Work Done

We completed all of the stories listed except for the file upload. This was mainly due to the fact that we were learning the Unity workflow with VR along with the Meta SDK. We worked in pair programming, as it was best for us to just sit down and knock each of the features out while also figuring out the proper design.

5.2.3 Testing Coverage



Coverage

Collapse all | Expand all Grouping: By assembly Compare with: Date Filter:

Name	Covered	Uncovered	Coverable	Total	Line coverage	Covered	Total	Branch coverage
Assembly-CSharp-Editor	**0**	**140**	**140**	**242**	**0%**	**0**	**0**	
ReadmeEditor	0	140	140	242	0%	0	0	
PuzzleGameScripts	**101**	**48**	**149**	**374**	**67.7%**	**0**	**0**	
Chunk	54	15	69	130	78.2%	0	0	
ChunkFactory	5	0	5	20	100%	0	0	
Piece	34	0	34	75	100%	0	0	
PieceMeshGenerator	8	0	8	55	100%	0	0	
Puzzle	0	7	7	19	0%	0	0	
PuzzleSeeder	0	26	26	75	0%	0	0	
PuzzleGameTests	**336**	**0**	**336**	**637**	**100%**	**0**	**0**	

Figure 9: Iteration 1 Testing Coverage

We think our testing is good, but would like to test some of the extra logic on the Chunk class. This was made difficult by the shorter iteration and we think with the extra time it would be good to make a few more unit tests for that class to be extra confident in its behavior. We did not test Puzzle or Puzzle Seeder because they are only used for manual testing on the actual VR headset, so we have at least 78% for each script that matters.

5.2.4 Retroespective & Reflection

Overall, we believe we did quite well on this iteration, especially considering the shorter timeframe. We learned a lot about Unity and how VR works. We also found that the Meta SDK is quite convenient because they have some good prototyping "building blocks." The only feature we had trouble with was the file upload, as we learned that it isn't as simple as it would be on other devices. The Meta Quest has a modified version Android as its OS. Because of this, it has the filesystem a bit locked down, and apps are sandboxed. We have to figure out whether it is even possible to upload a file from the local filesystem. We were exploring alternatives such as fetching an image via URL. It may be possible to find a way to upload local files from

the system, since it seems that we might be able to grant the app permissions. In total we spent a little more hours in the second week. The first was about 9, but the second was probably more like 12-15.

5.3 Iteration/Sprint 2

5.3.1 Planning

Team Member	Story	Story Description	Points
	U3	Jigsaw Piece Generation	8
Joe		**Joe Total**	**8**
	U4	Local Puzzle Creation	2
Kyle	U5	Local Puzzle Delete	2
	U14	Save Progress	2
	U22	Load Progress	2
		Kyle Total	**8**
		Total	**16**

Table 6: Iteration 2 Planning

5.3.2 Work Done

Team Member	Story	Story Description	Points
	U3	Jigsaw Piece Generation	6/8
Joe		**Joe Total**	**6/8**
	U4	Local Puzzle Creation	1.5/2
Kyle	U5	Local Puzzle Delete	1.5/2
	U14	Save Progress	1.5/2
	U22	Load Progress	1.5/2
		Kyle Total	**6/8**
		Total	**12/16**

Table 7: Iteration 2 Work Done

We have made progress on all the stories that we planned for iteration 2, and got them working in all other aspects except for the UIs, which is why we estimate them at 75% complete each. The UIs were more annoying than we initially thought, and we will need a little extra time to figure them out. Once we figure them all out though we should ideally get them done much faster in later iterations.

5.3.3 Testing Coverage

PuzzleGameScripts	**466**	**157**	**623**	**1341**	**74.7%**
Chunk	65	31	96	174	67.7%
ChunkFactory	5	9	14	41	35.7%
Events.PlayButtonBehaviour	0	8	8	25	0%
Persistence.LocalSave	51	8	59	101	86.4%
Persistence.LocalSaveInitializer	0	8	8	20	0%
Persistence.PuzzleSaveData	10	0	10	42	100%
Piece	43	3	46	103	93.4%
Puzzle	9	50	59	105	15.2%
PuzzleGeneration.Jigsaw.JigsawPieceBorder	7	0	7	18	100%
PuzzleGeneration.Jigsaw.JigsawPieceEdgeUtil	24	1	25	81	96%
PuzzleGeneration.Jigsaw.JigsawPuzzleGenerator	85	0	85	182	100%
PuzzleGeneration.PieceCut	6	0	6	22	100%
PuzzleGeneration.PieceMeshGenerator	118	4	122	198	96.7%
PuzzleGeneration.PuzzleLayout	7	0	7	23	100%
PuzzleGeneration.Rectangle.RectanglePuzzleGenerator	32	0	32	71	100%
PuzzleRenderData	4	0	4	9	100%
Seeders.PuzzleSeeder	0	35	35	126	0%

Figure 10: Iteration 2 Testing Coverage

We think that our testing coverage is good so far. The PuzzleSeeder is for manual testing, which is why we did not test it. Also, The Puzzle class is likely to change a lot in the future as its current interface is unstable. This class is for storing the current state of a puzzle as the user is solving it. We may decide to add and remove more fields to store in the future and since unit tests are tightly coupled to what they test, we would likely have to repeatedly change both the code and unit tests. Once the Puzzle becomes more stable in the future, then we will start writing unit tests. We are also finding that writing unit tests for Chunk is difficult, and it may signify that we need to rethink design. Part of this difficulty is due to the fact that it currently depends on both Piece and Puzzle. The dependency with Piece is natural, as Chunks are made up of Pieces. The dependency on Puzzle, which is its parent, feels more awkward. This is because when Chunks collide and combine, they need to let the Puzzle know this so that it can remove the destroyed chunk from its list of Chunks. There are alternatives such as using the Observer design pattern but this feels less honest as the only observer would be the single parent puzzle which makes use think that it is overkill. Unity obviously deals a lot with parent/child relationships so we think there should be a solution idiomatic to unity for this exact problem which we should look into.

5.3.4 Retroespective & Reflection

Overall, we think we did well in this iteration besides the challenges with the UIs. We probably should have looked more into how creating UIs worked early in the first week of this iteration. This would have been very beneficial as we could have realized how annoying they were earlier, which would have helped us make a better estimate on when we could finish them by. Also, the local save had some challenges as we want to store unity-specific objects in the database like Vector2. However, this is not directly supported by DBLite so we currently have a workaround solution that first uses Unity's serializer and then stores that serialized string in the BDLite database. We learned a lot about how rendering works in computer graphics. We learned about both the Bézier curve algorithms and Ear clipping algorithms, which are extremely common in computer graphics. The Bézier curve seemed complicated at first, but once we understood how it works and why it was used, it made a lot of sense to use it in this project. These curves are calculated from a list of control points, and are extremely common in computer graphics due to how easily scalable they are. For now, we just put the jigsaw socket in the middle of the piece's edge, and it is always the same size, but for future polishing, we can easily adjust the control points to move the socket along the edge or increase the size of it. Additionally, the ear clipping algorithm was very interesting and convenient because it works for

any outline of a simple polygon that does not overlap itself. For now, we just have the jigsaw and rectangle, which for the rectangle we just triangulate manually, as a rectangle face only needs 2 triangles, but the jigsaw shape would be extremely time-consuming and error-prone to triangulate manually, so the ear clipping was good here. Additionally, since it is so versatile, we can reuse it for other piece shapes in the future. However, there is the tradeoff of it being slow which may become an issue for larger puzzles but also possibly not since each puzzle piece really doesn't need too many vertices.

5.4 Iteration/Sprint 3

5.4.1 Planning

Team Member	Story	Story Description	Points
Joe	U24	Real Puzzle Piece Detection (Pair Programming)	4
	U25	Real Puzzle Piece Detection Report (Pair Programming)	1
	U26	Real Puzzle Piece Render (Pair Programming)	4
		Joe Total	**9**
Kyle	U24	Real Puzzle Piece Detection (Pair Programming)	4
	U25	Real Puzzle Piece Detection Report (Pair Programming)	1
	U26	Real Puzzle Piece Render (Pair Programming)	4
		Kyle Total	**9**
		Total	**18**

Table 8: Iteration 3 Planning

5.4.2 Work Done

Team Member	Story	Story Description	Points
Joe	U3	Jigsaw Piece Generation	2/8 (8/8 total)
	U22	Load Progress	0.5/2 (2/2 total)
	U24	Real Puzzle Piece Detection	6/8
	U26	Real Puzzle Piece Render	1/5
		Joe Total	**9.5**
Kyle	U4	Local Puzzle Creation	0.5/2 (2/2 total)
	U22	Load Progress	0.5/2 (2/2 total)
	U10	Puzzle Upload	1.5/2
	U11	Uploaded Puzzle Deletion	1.5/2
	U12	View Other Puzzle Uploads	1.5/2
	U13	Edit Uploaded Puzzles	1.5/2
	U27	User Auth	1.5/2
		Kyle Total	**8.5**
		Total	**18**

Table 9: Iteration 3 Work Done

We completed the stories that we had left over from last iteration. We have made significant progress on the stories for real puzzle generation. We created a python microservice on the backend to handle this task because it can be CPU intensive, so we think the more powerful server CPU will be much worth the trade off of having to send a request and send back a response to the client. So far, it works well for very accurate images with a consistent background and a small number of puzzle pieces that have distinct edge shapes. This is because we fit a spline around the perimeter of the piece so pieces with similar edges can easily be put in the wrong place. We are thinking about polishing this up in the future by checking that the final shape resembles what the puzzle should look like. For example if we can know ahead of time it is a rectangle (or maybe just support rectangles only), it is likely that connecting wrong pieces will produce a very different

shape than a rectangle so we can know they are wrong. Additionally, performance is still an issue because we compare many different orientations when comparing if different pieces fit together. Additionally, we do not think that Python is the main bottleneck because we try to offload the CPU intensive tasks to numpy and OpenCV which are implemented in C++.

In addition, we got ahead on some of the planned stories for Iteration 4, as we were unsure how complex the real puzzle generation would be. We finished much of the functionality for the online puzzles, however we have not added any UI for this due to the fact that setting up the server and meta quest auth was a large task on its own. Because of this along with the fact that we now have experience in the creation of UI, we estimate these stories to be 75% complete.

5.4.3 Testing Coverage

PuzzleGameScripts	620	164	784	1772	79%
Chunk	66	16	82	154	80.4%
Persistence.LocalSave	89	8	97	161	91.7%
Persistence.LocalSaveInitializer	0	8	8	20	0%
Persistence.PuzzleSaveData	11	0	11	46	100%
Piece	55	3	58	128	94.8%
PieceResponse	0	3	3	18	0%
Puzzle	30	21	51	102	58.8%
PuzzleGeneration.Jigsaw.JigsawPieceBorder	7	0	7	18	100%
PuzzleGeneration.Jigsaw.JigsawPieceEdgeUtil	24	1	25	82	96%
PuzzleGeneration.Jigsaw.JigsawPuzzleGenerator	94	0	94	204	100%
PuzzleGeneration.PieceCut	7	0	7	24	100%
PuzzleGeneration.PieceMeshGenerator	119	4	123	200	96.7%
PuzzleGeneration.PieceShapeExtensions	4	2	6	27	66.6%
PuzzleGeneration.PuzzleLayout	7	0	7	23	100%
PuzzleGeneration.Real.RealPuzzleGenerator	0	2	2	12	0%
PuzzleGeneration.Rectangle.RectanglePuzzleGenerator	37	0	37	80	100%
PuzzleManager	12	3	15	45	80%
PuzzleRenderData	7	0	7	14	100%
PuzzleResponse	0	16	16	40	0%
Seeders.PuzzleSeeder	0	42	42	142	0%
UI.HUD	6	3	9	29	66.6%
UI.PuzzleCreationBehaviour	13	18	31	82	41.9%
UI.PuzzleGallery	25	3	28	56	89.2%
UI.PuzzleGalleryTile	7	3	10	39	70%
UI.PuzzleInfo	0	8	8	26	0%

Figure 11: Iteration 3 Unity Testing Coverage

File	statements	missing	excluded	coverage
piece_response.py	31	0	0	100%
piece.py	152	6	0	96%
puzzle_response.py	12	0	0	100%
puzzle_solver.py	174	2	0	99%
puzzle.py	135	42	0	69%
tests / test_puzzle_response.py	18	0	0	100%
tests / test_puzzle_solver.py	41	7	0	83%
tests / test_transformation.py	40	0	0	100%
transformation.py	46	1	0	98%
Total	**649**	**58**	**0**	**91%**

Figure 12: Iteration 3 Real Puzzle Microservice Testing Coverage

Our coverage is good so far. There are a few files on the Unity side that have 0% in the coverage report because they will be changed significantly in the future. These files are related to the real puzzle import, the online puzzles, or just for debugging such as the PuzzleSeeder, so we are holding off on unit testing them for now. Additionally, even though we have high coverage on the real puzzle microservice side, it is important to note that we achieved this with only a few unit tests. We only have a few unit tests because manual testing is much more useful here. For example, it does not make any sense to write a unit test to verify that puzzle pieces in the image are correctly reconstructed because you need a human to verify that.

5.4.4 Retroespective & Reflection

Overall, we think we did well this iteration considering how worried we were about the real puzzle scanning. We were proud of the fact that we were able to learn about OpenCV and curve fitting. We also felt clever in creating the algorithm to score each of the socket pairings by snapping anchor points on the convex hull to the deepest concave points. One of the main issues we see is that there needs fine tuning, but we do not know exactly what is "good enough". For example, there is an error propagation issue where puzzle pieces will be slightly off. This will begin to add up and misalign the puzzle. As well as this, there is the issue of pieces which have a single edge touching multiple edges for different pieces. This is a pain point because it would be nice if every touching edge was the exact same length as that would greatly simplify the problem as we could just directly compare their points. Right now, we have tried out best to mitigate these issues, however there will always be error, so we need to find a good stopping point. We also were quite happy that we were able to get a lot of the online puzzle stories done, as we will have more time to work on the real puzzle and polish things up. We had a pain point with Meta's account authentication, as we had to create a team and test users using their system. However, we seemed to have figured it out properly, so with this we are hopeful that interacting with their API will be easier.

5.5 Iteration/Sprint 4

5.5.1 Planning

Team Member	Story	Story Description	Points
Joe	U26	Real Puzzle Piece Render	4/5 remaining
	U28	Support Other Piece Shapes	5
		Joe Total	**9**
Kyle	U10	Puzzle Upload	0.5/2 remaining
	U11	Uploaded Puzzle Deletion	0.5/2 remaining
	U12	View Other Puzzle Uploads	0.5/2 remaining
	U13	Edit Uploaded Puzzles	0.5/2 remaining
	U27	User Auth	6.5/8 remaining (Changed point value, more details below)
		Kyle Total	**8.5**
		Total	**17.5**

Table 10: Iteration 4 Planning

5.5.2 Work Done

Team Member	Story	Story Description	Points
Joe	U26	Real Puzzle Piece Render	4/5 (5/5 Total)
	U28	Support Other Piece Shapes	5
	U9	Solve Time	2
		Joe Total	**11**
Kyle	U10	Puzzle Upload	0.5/2 (2/2 Total)
	U12	View Other Puzzle Uploads	0.5/2 (2/2 Total)
	U27	User Auth	5.5/8 (7/8 Total)
		Kyle Total	**6.5**
		Total	**17.5**

Table 11: Iteration 4 Work Done

One story we underestimated was user authentication. The authentication has been increased to 8 points to reflect how much of a nightmare it is to work with. Originally, authentication seemed like an easy task with a clear documentation guide. However this quickly became much more difficult. Meta requires developers to do a Data Use Checkup if they want to be able to use their API to fetch user data. The only way around this is to use the API on test users. Once everything was set up, we tried to use the API on test users. When this did not work despite doing seemingly everything the documentation said, this then became a frantic search for anything or anybody that could help explain the problem we were having. The error codes given did not clearly explain what was wrong, and it seemed as though we were doing everything correctly. Because of this, it became a multi day problem where we had to scrutinize every part of the code. We think we solved the issue, however it involved a process of verifying with Meta that we are real people. Since verifying, we have not tested whether this has changed the result yet.

5.5.3 Testing Coverage

PuzzleGameScripts	**951**	**404**	**1355**	**3207**	**70.1%**
Chunk	69	13	82	156	84.1%
Networking.DTO.PuzzleMetadataListDTO	0	3	3	18	0%
Networking.MetaQuestAuthenticationManager	0	60	60	92	0%
Networking.PuzzleServerApi	0	84	84	162	0%
Networking.PuzzleServerApiInitializer	0	8	8	22	0%
Networking.Request.CreatePuzzleRequest	0	6	6	20	0%
Persistence.LocalSave	90	10	100	165	90%
Persistence.LocalSaveInitializer	0	8	8	20	0%
Persistence.PuzzleMetadata	0	12	12	40	0%
Persistence.PuzzleSaveData	19	2	21	63	90.4%
Piece	60	0	60	131	100%
PieceResponse	0	3	3	18	0%
Puzzle	41	21	62	128	66.1%
PuzzleGeneration.Hexagon.HexagonPuzzleGenerator	123	0	123	254	100%
PuzzleGeneration.Jigsaw.JigsawPieceBorder	7	0	7	18	100%
PuzzleGeneration.Jigsaw.JigsawPieceEdgeUtil	24	1	25	82	96%
PuzzleGeneration.Jigsaw.JigsawPuzzleGenerator	96	0	96	212	100%
PuzzleGeneration.PieceCut	7	0	7	24	100%
PuzzleGeneration.PieceMeshGenerator	119	4	123	200	96.7%
PuzzleGeneration.PieceShapeExtensions	4	5	9	35	44.4%
PuzzleGeneration.PuzzleGenerationDimensions	7	0	7	21	100%
PuzzleGeneration.PuzzleLayout	7	0	7	23	100%
PuzzleGeneration.Real.RealPuzzleGenerator	0	21	21	57	0%
PuzzleGeneration.Rectangle.RectanglePuzzleGenerator	39	0	39	90	100%
PuzzleGeneration.Triangle.TrianglePair	4	0	4	15	100%
PuzzleGeneration.Triangle.TrianglePuzzleGenerator	65	0	65	159	100%
PuzzleGenerationData	7	0	7	14	100%
PuzzleManager	18	0	18	52	100%
PuzzleResponse	0	14	14	39	0%
Seeders.PuzzleSeeder	0	5	5	142	0%
UI.CompletionDialog	6	9	15	46	40%
UI.HUD	13	22	35	80	37.1%
UI.PuzzleCreationBehaviour	23	15	38	131	60.5%
UI.PuzzleGallery	41	43	84	196	48.8%
UI.PuzzleGalleryTile	6	3	9	34	66.6%
UI.PuzzleInfo	16	12	28	81	57.1%
UI.PuzzleOnlineGalleryTile	0	9	9	45	0%
UI.UIManager	36	11	47	103	76.5%
UI.UIUtils	4	0	4	19	100%

Figure 13: Iteration 4 Unity Testing Coverage

File	statements	missing	excluded	coverage
piece_response.py	34	0	0	100%
piece.py	161	1	0	99%
puzzle_response.py	18	0	0	100%
puzzle_solver.py	210	13	0	94%
puzzle.py	143	42	0	71%
tests / test_piece.py	26	0	0	100%
tests / test_puzzle_response.py	18	0	0	100%
tests / test_puzzle_solver.py	41	7	0	83%
tests / test_puzzle.py	60	0	0	100%
tests / test_transformation.py	40	0	0	100%
transformation.py	46	1	0	98%
Total	**797**	**64**	**0**	**92%**

Figure 14: Iteration 4 Real Puzzle Microservice Testing Coverage

We did not get to writing unit tests for the Kotlin part of the backend in this iteration, but we will do so in iteration 5. The reason why it has not been as high priority as the Unity unit tests is that it is mostly just handling requests and updating a database at this point, so unit tests do not add much value. Unit testing the behavior of our code that runs in Unity, such as the logic for snapping pieces together, is much more valuable. We also did not get to testing the Networking module which we will do in iteration 5.

5.5.4 Retroespective & Reflection

The user story for generating other piece shapes was much easier than expected. We originally estimated this user story at 5 points because the algorithms would be different based on the shape but they really are not too much different. In fact, the jigsaw was definitely harder than the 2 shapes we added which were hexagons and triangles. The trickiest parts were making sure the edge pieces were the correct shape, maintaining a rectangle, and determining which pieces should be neighbors. Neighbors just means the user should be able to connect them as you can obviously not connect pieces on opposite sides of the puzzle even if you move them such that they are the correct relative distance apart. Overall, this was an extremely easy algorithm conceptually. The real puzzle render was also easier than expected as it was not fully working last iteration but the problems were not as severe as we expected. The main issues were related to accidentally swapping width and height dimensions and mixing up the y axis direction as it goes downwards when OpenCV draws contours but goes upwards in unity. This led to another issue of having the piece border point go counter clockwise instead of clockwise which was an issue when generating the mesh. Unity will only display faces whose points show in clockwise order so our ear clipping algorithm for triangulating the piece expects the points in clockwise order. Therefore, when giving it points in counter clockwise order it will fail to triangulate the piece. We may need to have more rigorous testing for the real puzzle microservice as we have now added

a test to see if the points are actually clockwise. Originally we did not spend much time at all on unit testing here because it is really only good to tell if a real puzzle is generated correctly by looking at it manually. This was clearly the most obvious test case missed and there may be others we should look into. These issues were fixed this iteration so now the real puzzle import works decently well for puzzles with a small number of pieces, a good quality image and very distinct shapes. It is not very robust overall though so we will try to polish it up in iteration 5.

There were several design decisions that we had to make this iteration. The first was what to do about the Real Puzzle generator since it requires sending a request rather than just generating the puzzle on the client side. This affected the IPuzzleGenerator interface as before it was designed to just be synchronous. However, this was not too much of an issue as we just changed it to return a Task to allow async instead which only required changing the existing generators in 1 place. Besides that the original generators run as normal and the real puzzle generator would just call the API. Either way, the generators all get the desired generated puzzle whether sending a request or not. Another design decision we had to make was how to manage the Puzzle and UI states. We decided to make a PuzzleManager class that would hold a reference to the current puzzle if one is currently being solved or null if not, and any other class that needed to know about the current puzzle would get it through the PuzzleManager. We also have a UIManager which handles which UI screens show up at which time. For example, when a user clicks on a puzzle in the gallery, it controls what should disappear and what should appear. We think the Manager classes work well and help us reason better about the logic. Probably one of the biggest design wins in our opinion was making use of the observer pattern. For example, the puzzle progress is updated, i.e. the user connects pieces together, we need to update the HUD that shows the progress and we also need to check to see if they have completed the puzzle and display a dialog if so. Right now, we have the UIManager subscribe an observer to when progress is updated to know when to display the competitor dialog screen. We also have the HUD subscribe so that it can know when to update the progress bar. Additionally, this design pattern is good because it decouples the Puzzle class from the UI. Having the Puzzle class call UI methods would definitely be a design smell as the Puzzle class is only supposed to be responsible for the Puzzle state. We use the observer in other places and will probably continue to because it is really useful.

One of the earlier iteration decisions we made that made something harder was the combining of a player's puzzle save data with the puzzle's metadata like the name and layout. For local puzzles this made a lot of sense since there was no need for separation. Local puzzles always only had one user so there would only be one user save data tied to the puzzle itself. Eventually, when the online puzzle CRUD became a feature that we actually needed to do, we realized this issue and tried separating the save data from the metadata, and then using a separate object named PuzzleData to encapsulate both. We started to do this and made a bunch of changes. However, we realized that even with the amount of changes made, there would still be so many more as the old data structure was passed practically everywhere. We realized that this would be an absolute nightmare change to make, because it would most certainly cause breakage everywhere in the application. The frontend, python microservice, and Kotlin service would all need changes. Overall we would want to fix this as soon as possible, but we also need features done. The problem comes in because now there is a massive technical debt that needs to be taken care of.

One thing we did well this iteration was bug fixing and polishing from last iteration. Our UIs were definitely a bit of a mess at the end of iteration 3. This iteration we did a much better job connecting them and having them interact with each other. However, we still have a few things that we would not be confident putting into production that we want to address in the future. First, the grab with the controllers seems to work well, but the hand grab interaction is still a bit buggy and we are not sure the reason why. Next, we also notice the issue that when you connect puzzle chunks together it is arbitrary when it chooses which one to rip out of your hand and connect to the other. We will talk again with our client about what his preferred behavior is, but something like having to let go of the piece to have it snap into place could be more natural and prevent the user from abusing the strategy of swinging the pieces around the screen to connect a bunch of them really fast. There is also the issue of the game freezing when creating a puzzle because it seems it blocks the main threads until it completes, which we want to fix in iteration 5.

Overall, we think we did a decent job with communication as we just worked on separate stories and avoided stepping on each other's toes rather than constantly changing the same code and having many different merge conflicts. We both had our tasks and understood what needed to be accomplished so we think things worked out well in that regard. Next time, we plan on budgeting time better as we fell a

little short in this category this iteration. It almost felt like a bit of a shock because iteration 3 was really due 3 weeks later but this was only 2. We had trouble making progress in the first week due to poor time management but did much better in the second week.

5.6 Iteration/Sprint 5

5.6.1 Planning

Team Member	Story	Story Description	Points
Joe	U2	2D Picture Upload	1
	U15	Provide Piece Hint	2
	U24	Real Puzzle Piece Detection	2/8 remaining
	U25	Real Puzzle Piece Detection Report	2
		Joe Total	**7**
Kyle	U11	Uploaded Puzzle Deletion	0.5/2 remaining
	U13	Edit Uploaded Puzzles	0.5/2 remaining
	U27	User Auth	1/8 remaining
	U17	Leader Board Creation	2
	U18	View Puzzle Leader Board	2
	U19	Leader Board Update	2
		Kyle Total	**8**
		Total	**15**

Table 12: Iteration 5 Planning

5.6.2 Work Done

Team Member	Story	Story Description	Points
Joe	U5	Local Puzzle Delete	2
	U15	Provide Piece Hint	2
	U25	Real Puzzle Piece Detection Report	2
		Joe Total	**6**
Kyle	U17	Leader Board Creation	2
	U18	View Puzzle Leader Board	2
	U19	Leader Board Update	2
		Kyle Total	**6**
		Total	**12**

Table 13: Iteration 5 Work Done

We got most of the points that we planned done but yet again we did not finish the user auth or the image upload. We have several times thought we have gotten almost there but then it doesn't work for reasons that we cannot explain. We did get some progress on the auth because we got it to work when testing it through the unity editor but when uploading the actual APK to the headset it does not work which does not make sense to us. We ended up going down a long debugging session on both the auth and image upload. We think we are getting closer to the solution but who really knows at this point with the amount of pain the headset is causing us.

5.6.3 Testing Coverage

PuzzleGameScripts	1258	765	2023	4778	62.1%
Chunk	64	0	64	131	100%
HintManager	63	6	69	121	91.3%
Networking.API.ContentApi	0	11	11	32	0%
Networking.API.MetaQuestAuthenticationManager	0	56	56	88	0%
Networking.API.PuzzleApi	0	43	43	103	0%
Networking.API.PuzzleServerApi	0	21	21	45	0%
Networking.DTO.JsonArray[T]	0	5	5	19	0%
Networking.LeaderboardApi	0	26	26	56	0%
Networking.PuzzleServerApiInitializer	0	8	8	23	0%
Networking.Request.CreatePuzzleRequest	0	5	5	18	0%
Persistence.LocalSave	92	10	102	170	90.1%
Persistence.LocalSaveInitializer	0	8	8	20	0%
Persistence.PuzzleMetadata	0	13	13	44	0%
Persistence.PuzzleSaveData	21	8	29	95	72.4%
Piece	91	12	103	209	88.3%
PieceMissingConnections	4	0	4	13	100%
PieceResponse	0	3	3	19	0%
Puzzle	113	21	134	251	84.3%
PuzzleGeneration.Hexagon.HexagonPuzzleGenerator	123	0	123	261	100%
PuzzleGeneration.Jigsaw.JigsawPieceBorder	7	0	7	18	100%
PuzzleGeneration.Jigsaw.JigsawPieceEdgeUtil	24	1	25	82	96%
PuzzleGeneration.Jigsaw.JigsawPuzzleGenerator	96	0	96	219	100%
PuzzleGeneration.PieceCut	7	0	7	29	100%
PuzzleGeneration.PieceMeshGenerator	120	4	124	202	96.7%
PuzzleGeneration.PieceShapeExtensions	4	5	9	35	44.4%
PuzzleGeneration.PuzzleGenerationDimensions	7	0	7	21	100%
PuzzleGeneration.PuzzleLayout	9	0	9	34	100%
PuzzleGeneration.Real.RealPuzzleGenerator	0	21	21	57	0%
PuzzleGeneration.Rectangle.RectanglePuzzleGenerator	39	0	39	89	100%
PuzzleGeneration.Triangle.TrianglePair	4	0	4	15	100%
PuzzleGeneration.Triangle.TrianglePuzzleGenerator	65	0	65	168	100%
PuzzleGenerationData	7	0	7	14	100%
PuzzleManager	39	40	79	151	49.3%
PuzzlePlacement	33	0	33	79	100%
PuzzleResponse	0	14	14	49	0%
Seeders.PuzzleSeeder	0	5	5	142	0%
UI.CompletionDialog	6	9	15	44	40%
UI.FileBrowserTest	0	21	21	101	0%
UI.FollowCamera	0	42	42	84	0%
UI.HUD	26	24	50	108	52%
UI.LeaderboardRow	0	5	5	21	0%
UI.PuzzleCreationBehaviour	14	15	29	88	48.2%
UI.PuzzleForm	8	0	8	23	100%
UI.PuzzleFormBehaviour	9	10	19	72	47.3%
UI.PuzzleGallery	47	65	112	239	41.9%
UI.PuzzleGalleryTile	6	9	15	51	40%
UI.PuzzleInfo	31	46	77	191	40.2%
UI.PuzzleLeaderboard	0	22	22	52	0%
UI.PuzzleOnlineGalleryTile	0	9	9	39	0%
UI.PuzzleSettings	0	59	59	158	0%
UI.RealPuzzleDetectionReport	0	21	21	68	0%
UI.UIManager	44	55	99	206	44.4%
UI.UIUtils	13	7	20	74	65%
Util	22	0	22	37	100%

Figure 15: Iteration 5 Unity Testing Coverage

Coverage report: 92%

Files | Functions | Classes

coverage.py v7.13.4, created at 2026-04-15 01:58 -0400

File	statements	missing	excluded	coverage
piece_response.py	34	0	0	100%
piece.py	161	1	0	99%
puzzle_response.py	40	0	0	100%
puzzle_solver.py	223	13	0	94%
puzzle.py	140	42	0	70%
tests / test_piece.py	26	0	0	100%
tests / test_puzzle_response.py	18	0	0	100%
tests / test_puzzle_solver.py	41	7	0	83%
tests / test_puzzle.py	60	0	0	100%
tests / test_transformation.py	40	0	0	100%
transformation.py	46	1	0	98%
Total	**829**	**64**	**0**	**92%**

Figure 16: Iteration 5 Real Puzzle Microservice Testing Coverage

We did not get to testing the Kotlin side of the server as originally planned as polishing took a very long time this iteration as we kept discovering subtle bugs and then reintroducing bugs when trying to polish. We also spent a lot of time trying to figure out the auth issues and the image upload issues since these are important features. Also, the Kotlin side of the backend is mostly just receiving requests, updating a database and sending responses which is not too terrible to just verify it is working by manual testing which is why we pushed off making unit tests and ended up running out of time. We also did not get to testing the C# frontend side of sending requests to this server. We did make sure to do a good job testing the new changes to the Puzzle, Chunk, and Piece classes as this is the most important part of the game and we need to make sure that behavior is working correctly. Could definitley do better as we only got to 62.1% on the Unity side.

5.6.4 Retroespective & Reflection

During this iteration we made use of the chain of responsibility design pattern. This for example made the logic of determining if pieces should connect with each other much cleaner especially when using it in combination with the observer pattern. For example there is the challenge of the fact that the Puzzle Class

needs to know about the Chunk class as its children are Chunks, and similarly Chunk should know about the Piece class because its children are Pieces. However, when we need to check if pieces connect together we must check at the Piece level if they are close enough together, and then would have to notify the Chunk to update its list of Pieces and then let the puzzle know that one of its chunks has changed. This makes the design challenging because we felt that making a bidirectional dependency is a smell and would have made the code much more complicated. The last iteration we had a pretty hacky solution of making a bounding box around each chunk and checking if we should merge the chunks if their bounding boxes collide and they are close enough together and actually connect together. We would then use the observer pattern when the Chunks should be merged to bubble up to the Puzzle class. Technically using the observer pattern to bubble up to the Puzzle does solve the bidirectional dependency problem but it was still the architecturally wrong solution.The reason for this is that not only is it hacky but the logic also starts at the piece level. The chain is a piece is dropped, which notifies its chunk, and the chunk notifies the puzzle. The puzzle then checks if it is relatively close enough to another chunk and merges them if it is. This also fixed the design smell of having the chunk handle all the merge logic and deleting the other chunk it merges with. It makes a lot more sense for the Puzzle to handle merging and deleting chunks since it is the parent of the chunks.

We also made some new abstractions and removed some duplicate code as a result. For example, the puzzle creation and puzzle settings screen, which allows you to edit the puzzle, we made a shared form for both of them rather than duplicating the logic for a bunch of the same fields.

A design decision that we think was not great though was how we handle the online and local puzzles. We have a few different scattered if statements checking if a puzzle is local or online. This does not seem to scale very well but we wanted to prioritize just getting things working rather than making perfect design. Probably what we should have done was make an interface which would be implemented by both local and online components, but we would have to think more about exactly how to design this.

During this iteration there were several bugs discovered and introduced as we worked on polishing previous features. There were a lot of UI state bugs which showed up when switching between UI screens. For example, we had the issue of UI screens filling in the previous puzzle's data. It worked for local but not online puzzles, so when you played a local puzzle and then an online puzzle the gameplay hud would show stats related to the local puzzle incorrectly.

One of the pitfalls with this iteration was unit testing as we had to go back and fix unit tests and we feel that we are not doing them completely correctly because we are using reflection in a lot of places to allow access to private fields. We are not sure what is the best way to handle this because it is a common pattern in unity to make private fields for the dependencies of an object and then set them in the editor. Would could technically just make them public but that also seems wrong to make them public so that we can access them in unit tests without using reflection. They do not need to be public otherwise so this wouldn't make sense. A better approach that we are thinking of is making use of dependency injection to pass the fields in from another place. This would also allow us to make the dependencies implement an interface so that for unit testing we could pass in a fake dependency that implements that interface, which is talked about here: https://softwareengineering.stackexchange.com/questions/140992/is-dependency-injection-essential-for-unit-testing.

Probably the most interesting thing we learned this iteration was using shaders in unity, which was used for highlighting the puzzle pieces when giving a hint. The way our shader worked was it made a highlight that expands a small amount around the border of the piece, which we can do this by calculating the centroid of the piece and expanding. We also make the image on the piece pulse opacity. We can see how these shaders can be really useful for making Unity games and would like to learn how they are typically used in other scenarios.

We think that overall we did well communicating as a team this iteration as we each handled our individual story points and did a good job avoiding stepping on each other's toes. One thing that we keep repeatedly doing is underestimating the amount of time it takes to complete these stories as there is always something annoying that shows up when programming a game for the quests that would not happen if we were just making a simple website like SWE last semester. Both the user authentication and image upload seemed really easy from the start for example and yet we have still not gotten them fully working.

6 Final Remarks

6.1 Overall Progress

We estimate that we would need 2 more iterations to get all the features our client wanted. We believe that we are very close on the user authentication and image upload stories, so we think that these could be completed within an iteration. We think we could get the image taking and cropping done in this same iteration. We would also want to spend an iteration improving the performance and polishing the real puzzle generation. We could easily see the real puzzle taking a whole extra iteration because of how difficult this problem is, and the original point estimate was likely way too low. We probably should have split this up into even more smaller user stories. Our client said he does not care about 3D puzzles anymore even though they would be cool, so we are not factoring this in.

6.2 Project Reflection

We have learned a lot more about the software development project, especially after getting experience doing a harder project than the software engineering class. This is especially true for the requirements-gathering stage because there were several things that we did not know how to make beforehand and were hard to estimate the point values for. This was not as true last semester because everyone in our group had experience with web programming and had a much better estimate of how long things would take. This time was different, especially dealing with the new environment of programming in Unity for the meta quest. Neither of us has done a project in Unity or made a game for the Meta Quest. Looking back on this, we probably should have done some research on how quest development works before assigning story point values. At the beginning of the semester when showing the class our story point values, Dr. Isaacman asked us how we went about choosing the point values, and we did not have a better answer other than just saying what seemed to be the difficulty. Doing more research beforehand could have helped us realize that the image upload was more difficult than we originally thought because of the file system limitations on the quest. Overall we both agree that the requirements phase is the most important phase of software development. The reason for this is that we cannot do anything else right if we do not get the requirements right, as it is not always a question of whether we build the product right. It has been a lot of time: did we build the right product? Also, getting right could avoid a lot of the point misestimations that we described above. Definitely a close second to requirements gathering is design. The design phase, especially early on, can really affect the progress made in future iterations. We found that the design smells we made early on did not really hurt us until later. For example, combining the meta and save data for puzzles was actually completely fine early on because we were only supporting local puzzles. So we only really regretted this when we made online puzzles and realized this design was not fine and were just not thinking about future consequences earlier.

For specific stories where we overestimated the amount of points supporting other piece shapes than jigsaw. We originally estimated this at 5 because the algorithms were likely going to be different based on the piece shapes, which they were but not by all that much. In fact, the Jigsaw was definitely more complicated than each of the ones added later. The user authentication and image uploaded were way underestimated. The user authentication was definitely hard to predict that this would happen because, according to Meta's documentation, it seemed easy. We ended up changing this story from 2 to 8 points to reflect the level of debugging it took. In the end it only works in the editor but still fails for some reason when we upload the build to the actual headset. For the image upload, this could have been avoided by doing more research at the beginning of the project. We would have found out pretty quickly that the file system limitations made this more difficult than our originally estimated 1 story point.

The requirements for this project stayed relatively constant over time. There were not any new requirements added after the project started; however, our client told us which features were a higher priority in general. For example, our client told us that he was more interested in 2D puzzles and did not care about 3D puzzles much and just considered them a nice-to-have. Therefore, we focused on the 2D to get it working well.

For the software design we learned more about how to make use of design patterns. We found the observer design pattern to be useful in many places such as updating UI elements when events happen. For example, when the user creates a new local puzzle, we want to add it to the grid of puzzles. Additionally, the user

can only connect pieces after dropping at least 1 of them, so we need to bubble up this event to the piece's chunk, and the chunk will bubble up this event to the puzzle so that it can connect the pieces if they are placed in the correct relative locations. After this experience, we think we are better at figuring out where these design patterns should be used and when they should not be used. Looking back, we can also identify instances where we should have used certain design patterns. For example, we think it could have been useful to apply the state design pattern, or something similar, to how we manage the UI. In the state where the user is just looking at all the puzzles, we would display the puzzle gallery, but when they click on a specific puzzle, it would transition to that puzzle's info screen, and in that state you would be able to do things like click on the settings menu or the leaderboard.

We had to learn several new tools for this project. The main one was Unity, as neither of us has programmed a game in Unity before. Unity is a massive framework, so it was overwhelming at first, but we only ended up needing a very small subset of the tools anyways. Unity uses C#, which one of us has not used before, but this one was not too difficult to learn, as it is really similar to Java. We have also used a bit of OpenCV in the past but learned a lot more about the useful functions it provides that made the real puzzle import significantly easier. For example, it handles the math of finding the convex hull and the convexity defects, which was very useful for our strategy of finding which pieces connect where.

For ensuring internal quality of software, we learned that while unit testing is good for testing correctness of small pieces of code, we found it to also be useful for exposing bad design decisions. For example, we found ourselves using reflection to access private fields in unit tests, which was probably not the best approach, and looking back on it now, we would have relied more on dependency injection. For this specific project, though, we found manual testing to be more useful than unit testing in many cases. The reason for this is it was hard to actually verify that a lot of the logic is correct without actually playing the game and getting a feel for it. For example, we wanted to make sure that the piece connection was clean when a user drops a piece into place. It is not really possible to ensure this feels right from a user's perspective without manual testing. Additionally, when testing the real puzzle reconstruction, it is hard to make unit tests that verify a puzzle is reconstructed correctly, but it is easy to look at an image of it to tell.

For this project we used Unity's unit testing framework, which is based on the NUnit framework. It was quite easy using this unit testing tool, as it is very similar to unit testing frameworks that we have used in the past. Similar to the testing tool that we used with Python, as we just used the unit testing framework in its standard library.

We tried to take steps to make our software maintainable, especially in the beginning of the project when we had a much smaller codebase. We are glad we made design decisions such as making the puzzle generator interface and making use of the observer design pattern. However, towards the end of the project, we started just living with design smells as we wanted to prioritize pushing out features quickly. We wish we had used more interfaces, though, as they would have helped for the local vs. online puzzle problem and would have made testing easier because we would have substituted dependencies for mocks that implement the same interface.

We used AI for a few different purposes in this project. Some of the setup and boilerplate was useful to use AI to help with. For example, for the real puzzle import, it was helpful in showing how to make basic API endpoints in Python's FastAPI. However, it was not as useful in setting up things related specifically to the quest. For example, when we had a build fail, it suggested switching the build target to Windows, which makes no sense because the Quest does not even run on Windows. We also used AI for boilerplate in unit tests such as destroying objects on teardown. Additionally, we used it for math formulas, which included how to use the quaternions to rotate vectors in Unity. It was also used for the math of how to apply transformations to puzzle pieces to fit them into place so the resulting image of the reconstructed puzzle looks correct after we figured out which pieces go together and where.

Appendix

[Appendix section if needed]